

OLYMPICS PLAYER MANAGEMENT SYSTEM

PROJECT REPORT

Student Name	NIDHIN B S
Registration Number	26BCE10789
Project Type	Python Modular Application
Storage	olympics.csv
Core Concepts	Modules, Functions, CSV, CRUD, Sorting

Submitted as part of the VITYarathi flipped-course project evaluation.

1. Introduction

The Olympics Player Management System is a menu-driven Python application for maintaining player records and medal information. It stores records in a CSV file and separates major operations into individual Python modules. The system supports adding players, searching records, updating medal counts, removing players, viewing players by points, and country-wise medal analysis.

The VITyarthi guideline asks students to identify a meaningful problem, design a technical solution, implement it using course concepts, and demonstrate understanding through documentation and evaluation. ■filecite■turn0file0■L14-L20■

2. Problem Statement

Manual management of player and medal information becomes difficult as records increase. The proposed system provides a simple computerized method to store player information and perform insertion, searching, updating, deletion, ranking, and country-wise aggregation.

Target users: Students or small-scale sports-data administrators.

Scope: Local Python application using CSV storage. No database server, web interface, authentication, or multi-user networking is included in the current implementation.

3. Objectives

- Create a modular Python application for managing player records.
- Store player and medal information persistently in olympics.csv.
- Provide add, search, update, and remove operations.
- Calculate points using $\text{Gold} \times 5 + \text{Silver} \times 3 + \text{Bronze} \times 1$.
- Display players in descending order of points.
- Calculate country-wise medal totals and points.
- Demonstrate functions, modules, loops, lists, sorting, and file handling.

4. Functional Requirements

Module	Function	Input	Result
Player Addition	addplayer()	ID, name, country, item, medals	Record added; points calculated
Player Search	search()	Player ID	Matching record displayed
Prize Update	prizeupdate()	ID + medal additions	Medals and points updated
Player Removal	remove()	Player ID	Record removed
Player View	playertop()	None	Players sorted by points
Country View	topcountry()	None	Country totals displayed

5. Non-Functional Requirements

Requirement	Implementation
Usability	Numbered menu with clear prompts.
Reliability	Persistent CSV storage and checks for missing/duplicate IDs.
Maintainability	Separate Python module for each major operation.
Resource Efficiency	Lightweight CSV storage and standard Python libraries.
Error Handling	Invalid menu choices and player IDs are handled with messages.
Scalability	Modular design allows extension, though CSV is intended for small datasets.

The project guideline requires at least four non-functional requirements and lists performance, security, usability, reliability, scalability, maintainability, error handling, logging, and resource efficiency as examples. ■filecite■turn0file0■L38-L50■

6. System Architecture

The system follows a simple modular architecture. main.py controls the user menu and imports the operation modules. The modules read or rewrite the shared olympics.csv file.

Architecture: User → main.py → selected module → olympics.csv → result displayed to user.

Project structure

olympics-Player-Management/
 ■■■ main.py
 ■■■ playeradd.py
 ■■■ searchplayer.py
 ■■■ prizeupdation.py
 ■■■ removeplayer.py
 ■■■ topplayer.py
 ■■■ country.py
 ■■■ olympics.csv

The supplied project contains the main program plus six operation modules, fitting the guideline's expectation of 5–10 meaningful modules/classes/files for coding projects. ■filecite■turn0file0■L51-L63■

7. Design Diagrams

7.1 Use Case Diagram

USER	OLYMPICS PLAYER MANAGEMENT SYSTEM
User	Add Player Search Player Update Prize
	Remove Player View Players View Countries

7.2 Workflow Diagram

START	→	DISPLAY MENU	→	READ CHOICE
		↓		
	←	CONTINUE?	←	EXECUTE SELECTED MODULE
		↓		
	←	EXIT	←	INVALID CHOICE / MESSAGE

7.3 Sequence Diagram

User	Main Menu	Module	CSV File
Select choice →			
	Call function →		
		Read / write →	
		← Return result	
← Display result			

7.4 Class / Component Diagram

main.py	playeradd.py	searchplayer.py
Menu + loop	addplayer()	search()
prizeupdatetime.py	removeplayer.py	topplayer.py
prizeupdate()	remove()	playertop()
		country.py → topcountry()
		↓
	olympics.csv	

7.5 Storage / ER Representation

A conventional ER diagram is not directly applicable because the implementation uses a CSV file. The logical record schema is: player_id, player_name, country, item, gold, silver, bronze, points.

8. Design Decisions & Rationale

- **CSV storage:** Simple and suitable for a small educational application.
- **Modular functions:** Each operation is isolated in a separate file.
- **Menu-driven interface:** Easy console-based interaction.
- **Derived points:** Points are calculated from medal counts to avoid separate manual entry.
- **Sorting:** Player and country totals are sorted in descending order for display.

9. Implementation Details

9.1 Main Menu

```
main.py
from playeradd import addplayer
from searchplayer import search
from prizeupdatetime import prizeupdate
from removeplayer import remove
from topplayer import playertop
from country import topcountry

# Menu options 1 to 6 call the selected function.
```

9.2 Point Calculation

```
Medal scoring
points = (int(gold) * 5) + (int(silver) * 3) + (int(bronze) * 1)
```

9.3 Add Player

addplayer() checks the player ID, accepts player details and medal counts, calculates points, and writes the record to olympics.csv.

9.4 Search Player

search() reads the CSV, checks whether the requested player ID exists, and prints the matching record.

9.5 Update Prize

prizeupdate() adds new medal counts to the existing values, recalculates points, and rewrites the CSV.

9.6 Remove Player

remove() keeps all records except the selected player and rewrites the CSV.

9.7 Ranking and Country Analysis

playertop() sorts player points in descending order. topcountry() aggregates medal counts and points for each country and sorts the country totals.

10. Screenshots / Results

10.1 Sample Dataset

ID	Name	Country	Item	G	S	B	Points
101	prarthana	pakistan	rugby	1	1	0	8
103	NIDHIN	india	football	1	1	1	9
104	livin	india	cricket	1	0	1	6
105	messi	argentina	football	1	0	1	6

10.2 Representative Console Result

Search

```
enter the choice mentioned above: 2
enter the player_id: 103
```

Details of the player searched

```
103 :: NIDHIN :: india :: football :: 1 :: 1 :: 1 :: 9
```

Country totals

```
country :: gold :: silver :: bronze :: points
india :: 2 :: 1 :: 2 :: 15
pakistan :: 1 :: 1 :: 0 :: 8
argentina :: 1 :: 0 :: 1 :: 6
```

These are representative results calculated from the supplied CSV data and code logic; they are not screenshots from an executed terminal session.

11. Testing Approach

ID	Operation	Input	Expected Result
T01	Add	New ID 106	New row and calculated points
T02	Duplicate	ID 103	Duplicate rejected
T03	Search	103	NIDHIN record displayed
T04	Invalid search	999	Player not found
T05	Update	103 + medals	Medals and points updated
T06	Remove	105	Record removed
T07	View players	5	Descending point order
T08	View countries	6	Country totals displayed
T09	Invalid menu	9	Valid-choice message

The guideline specifically calls for testing wherever applicable and validation/error handling.

■filecite■turn0file0■L51-L63■

12. Challenges Faced

- Maintaining a consistent CSV structure during updates and deletions.
- Preventing duplicate player IDs.
- Recalculating points whenever medal counts change.
- Aggregating country totals when multiple players belong to one country.
- Keeping the main program simple while distributing functionality across modules.

13. Learnings & Key Takeaways

- Modular Python programming and reusable functions.
- CSV reading and writing with the csv module.
- CRUD-style operations on persistent data.
- Use of loops, conditions, lists, and sorting.
- Calculation of derived values from stored attributes.
- Importance of validation, testing, maintainability, and documentation.

14. Future Enhancements

- Replace CSV with SQLite/MySQL.
 - Add GUI or web interface.
 - Add stronger numeric and text validation.
 - Add authentication and role-based access.
 - Add automated unit tests.
 - Add charts for medal and country statistics.
 - Add GitHub README.md and statement.md according to the submission requirements.
- filecite■turn0file0■L87-L106■

15. Conclusion

The Olympics Player Management System is a practical modular Python project for managing player and medal records. It implements insertion, searching, updating, deletion, ranking, and country-wise aggregation using CSV storage. The project demonstrates core programming concepts through a real-world-style application and provides a clear base for future database, validation, testing, and interface improvements.

The report follows the requested report structure covering introduction, problem statement, requirements, architecture, design diagrams, implementation, results, testing, challenges, learnings, future enhancements, and references.

■filecite■turn0file0■L109-L134■

16. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (uploaded PDF).
- Python Standard Library concepts used by the project, especially csv and file handling.
- Python programming concepts used in the supplied source code: functions, modules, loops, conditional statements, lists, sorting, and file operations.